

## IOT Assignment

Name-Tanish Sharma

Q1) You are building a smart irrigation system that must monitor soil moisture, control a water pump, and send data to a mobile application every 5 seconds. You have both an Arduino UNO and an ESP32. Which controller would you choose and why? Would your answer change if internet connectivity was not required?

Q2) A smart factory contains 500 sensors distributed across a 2 km area. Compare Wi-Fi, Bluetooth, LoRa, and MQTT-based communication. Which protocol or combination of protocols would you choose and why?

Q3) A temperature sensor suddenly starts showing impossible values (e.g.,  $-120^{\circ}\text{C}$  or  $300^{\circ}\text{C}$ ). How you would determine whether the problem lies in the sensor, wiring, power supply, ADC, or software.

Q4) An ESP32-based weather station must operate on battery power for six months without maintenance. What hardware and software modifications would you make to maximize battery life?

Q5) An automatic braking system in a vehicle requires communication with almost zero delay. Which communication method would you choose and why?

Q6) You need to measure the average temperature of a classroom using only one sensor. Where would you place the sensor and why? Explain how poor placement could affect the accuracy of your measurements. Assume the classroom to be a square-base cuboid of side length “a” and height “b”.

Q7) Under what situations could an analog sensor perform better than a digital one?

Q8) Draw the complete architecture/block diagram of your IoT project, beginning from the sensor and ending at the user interface. Identify every possible point where a failure could occur and suggest one solution for each.

Q9) If an attacker gains access to your IoT device through the Wi-Fi network, what kinds of attacks could they perform? Suggest at least five methods to improve the security of the system (with applicable solutions, do not cite far-fetched ideas)

Q10) Your ESP32 must simultaneously interface with a DHT11 sensor, an ultrasonic sensor, an OLED display, a relay module, and a soil moisture sensor.

What hardware or software conflicts could occur, and how would you solve them?

Q11) An ESP32 uploads sensor data to the cloud every minute. The internet connection fails for 12 hours. Design a strategy that ensures no important data is permanently lost.

Q12) Your IoT device must operate continuously for five years without human intervention. What changes would you make to both the hardware and software to improve long-term reliability?

Q13) Can an IoT system be considered "smart" if it does not use Artificial Intelligence? Justify your answer with suitable examples. What differentiates "smart" from "intelligent"?

Q14) Compare polling and interrupt-based programming in embedded systems. In which situations would interrupts significantly improve the performance of an IoT device?

Q15) Every engineering design involves trade-offs. Looking at your own IoT project, identify three design decisions where you sacrificed one aspect (such as cost, speed, power, accuracy, or complexity) to improve another. Explain whether you believe those trade-offs were justified.

Q16) Create a flowchart for an IoT system that stores sensor readings locally whenever Wi-Fi is unavailable and automatically uploads the stored data once the connection is restored. Mention all the programs/protocols/I-O used in the system.

Q17) Draw a flowchart showing how an ESP32 performs an OTA (Over-the-Air) firmware update, including error handling if the update fails.

Answers 1: I would choose the **ESP32** because it has built-in Wi-Fi and Bluetooth, making it ideal for IoT applications. It has a faster dual-core processor (up to 240 MHz), more RAM, more Flash memory, multiple ADC channels, and supports multitasking using FreeRTOS. Since the system needs to upload data to a mobile application every 5 seconds while simultaneously monitoring soil moisture and controlling a water pump, the ESP32 can perform all these tasks efficiently.

If internet connectivity is **not required**, then **Arduino UNO** would be sufficient. It can easily read the soil moisture sensor and control the relay for the water pump. It is cheaper, simpler to program, and consumes less power than the ESP32. Therefore, ESP32 is the better choice for IoT-enabled irrigation, while Arduino UNO is suitable for standalone irrigation systems.

---

## Answer:2

| Protocol  | Range               | Speed              | Power Consumption | Advantages                      | Disadvantages                 |
|-----------|---------------------|--------------------|-------------------|---------------------------------|-------------------------------|
| Wi-Fi     | 50–100 m            | High               | High              | Fast data transfer              | Limited range                 |
| Bluetooth | 10–100 m            | Medium             | Low               | Low power                       | Short range                   |
| LoRa      | 2–15 km             | Low                | Very Low          | Long range, battery efficient   | Low data rate                 |
| MQTT      | Depends on Internet | Depends on Network | Low               | Lightweight cloud communication | Requires network connectivity |

For a factory with **500 sensors spread across 2 km**, the best solution is a **combination of LoRa and MQTT**. Sensors communicate with a LoRa gateway due to its long-range and low-power characteristics. The gateway then sends data to the cloud using Wi-Fi or Ethernet, and MQTT is used as the communication protocol between the gateway and cloud server. This combination provides reliable, scalable, and energy-efficient communication.

---

## Answer:3

If the sensor displays impossible values such as **−120°C or 300°C**, I would troubleshoot systematically.

First, I would inspect the **sensor** itself by comparing its readings with another calibrated sensor. If the problem persists, I would replace the sensor.

Next, I would check the **wiring** for loose connections, broken wires, incorrect pin connections, or corrosion.

Then, I would verify the **power supply** using a multimeter to ensure that the sensor receives the correct operating voltage (3.3V or 5V).

After that, I would examine the **ADC** by reading raw ADC values and applying a known input voltage to verify whether the analog-to-digital converter is functioning correctly.

Finally, I would inspect the **software**, checking the conversion equations, calibration constants, sensor library, and data types to ensure that no programming errors are producing incorrect values.

By testing each component individually, the actual source of the fault can be identified.

---

## Answer:4

To operate continuously for six months on battery power, I would make several hardware and software modifications.

On the hardware side, I would use a high-capacity lithium-ion battery, a low-quiescent current voltage regulator, and low-power sensors. If possible, I would include a small solar panel for battery charging.

On the software side, I would enable the ESP32 Deep Sleep Mode, allowing the microcontroller to wake only at fixed intervals (such as every 10 minutes). Wi-Fi would remain disabled except during data transmission. Bluetooth would also remain disabled. Sensor readings would be averaged before transmission to reduce communication frequency. The OLED display would remain off except when necessary.

These modifications significantly reduce energy consumption, allowing the system to operate for several months.

---

#### **Answer:5**

I would choose the **CAN (Controller Area Network) Bus** because it provides deterministic communication with extremely low latency and high reliability. CAN Bus supports automatic error detection, message prioritization, and fault tolerance, making it the standard communication protocol in modern vehicles. Wireless technologies such as Wi-Fi and Bluetooth introduce unpredictable delays and are therefore unsuitable for safety-critical applications like automatic braking.

---

#### **Answer:6**

The temperature sensor should be placed near the **center of the classroom**, approximately **1.2–1.5 meters above the floor**, away from windows, doors, air conditioners, fans, heaters, and direct sunlight. Since the classroom is assumed to be a square-base cuboid of side length **a** and height **b**, placing the sensor near the geometric center provides the best approximation of the average room temperature.

Poor placement can significantly affect accuracy. A sensor near a window may record lower temperatures during winter, while one near a heater or sunlight may record higher temperatures. Placement close to an air conditioner may produce readings much lower than the actual classroom average.

---

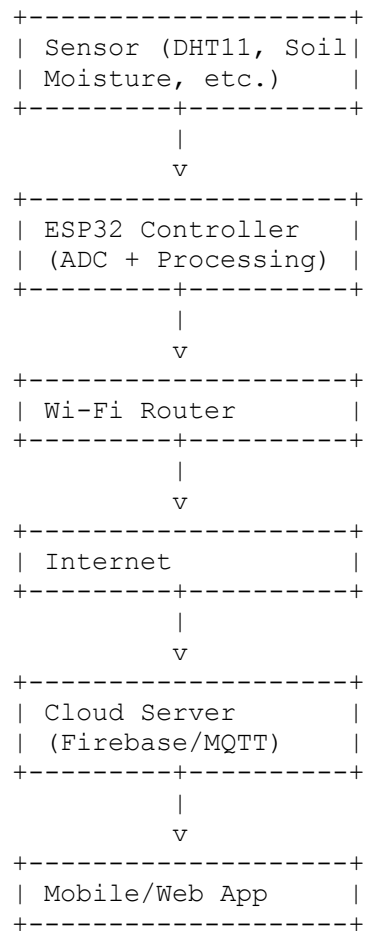
#### **Answer:7**

Analog sensors perform better when continuous measurements with fine resolution are required. They are generally less expensive, have faster response times, and produce continuous voltage outputs that accurately represent small physical changes.

Examples include thermistors, strain gauges, light-dependent resistors (LDRs), and microphones. Digital sensors are preferred when high noise immunity, built-in calibration, and direct digital communication are required.

---

## Answer8:



### Possible Failure Points and Solutions

| Failure Point | Possible Cause         | Solution                            |
|---------------|------------------------|-------------------------------------|
| Sensor        | Sensor damage or aging | Replace sensor, calibration         |
| Wiring        | Loose/broken wires     | Use proper connectors and shielding |
| ESP32         | Software crash         | Watchdog timer, restart mechanism   |
| Power Supply  | Battery drained        | Battery monitoring, backup battery  |
| Wi-Fi         | Network unavailable    | Store data locally and retry        |
| Internet      | ISP failure            | Offline data logging                |
| Cloud Server  | Server downtime        | Backup server, retry mechanism      |
| Mobile App    | Application crash      | Error handling and notifications    |

---

## Ans 9:

### Possible Attacks

1. Unauthorized access to ESP32
2. Sensor data theft
3. False sensor data injection
4. Malware installation
5. Denial-of-Service (DoS) attack
6. Pump/relay control by attacker
7. Password theft

## Methods to Improve Security

1. **Use WPA2/WPA3** secured Wi-Fi.
2. **Encrypt communication using HTTPS/TLS.**
3. **Use strong passwords** and change default credentials.
4. **Implement Secure Boot** to prevent unauthorized firmware.
5. **Use signed OTA firmware updates.**
6. Enable **user authentication** before accessing the device.
7. Disable unused services and ports.
8. Store passwords securely in encrypted memory.

---

Answer 10:

## Possible Conflicts

- GPIO pin shortage.
- Multiple devices trying to use the same communication bus.
- ADC readings affected by Wi-Fi activity.
- Blocking code delaying sensor readings.
- Timing conflicts with ultrasonic sensor.
- Relay switching introducing electrical noise.

## Solutions

- Plan GPIO assignments carefully.
- Use the **I<sup>2</sup>C bus** for OLED and other compatible devices.
- Use separate ADC pins for analog sensors.
- Implement **FreeRTOS tasks** for multitasking.
- Use interrupts for ultrasonic echo detection.
- Add flyback diodes and proper power filtering for relay circuits.

---

# Answer 11:

## Strategy

1. Read sensor data every minute.
2. Add timestamp to each reading.
3. Store readings in Flash memory or an SD card whenever Wi-Fi is unavailable.

4. Periodically check internet connectivity.
5. Once Wi-Fi reconnects, upload all stored data in chronological order.
6. Delete data from memory only after successful upload.
7. Resume normal operation.

This ensures **zero permanent data loss**.

---

## Answer 12:

### Hardware Improvements

- Industrial-grade ESP32 module
- High-quality sensors
- Surge protection
- Stable power supply
- Battery backup
- Waterproof enclosure (IP65/IP67)
- Corrosion-resistant PCB

### Software Improvements

- Watchdog timer
- Automatic restart after crashes
- OTA firmware updates
- Error logging
- Memory leak prevention
- Data backup
- Sensor calibration routines
- Self-diagnostic tests

These improvements increase reliability and reduce maintenance.

---

## Answer 13:

An IoT system **can be smart without Artificial Intelligence**.

For example, an automatic street light uses an LDR sensor to detect darkness and switches ON the light automatically. It follows predefined rules and does not learn from previous data.

An **intelligent** system uses Artificial Intelligence or Machine Learning to learn from historical data and improve its decisions over time.

### Difference

| Smart System           | Intelligent System         |
|------------------------|----------------------------|
| Rule-based             | Learns from data           |
| No learning            | Uses AI/ML                 |
| Fixed behaviour        | Adaptive behaviour         |
| Simple decision making | Predictive decision making |

#### Examples:

- Smart irrigation timer → Smart
- AI-based irrigation that predicts watering based on weather → Intelligent

## Answer 14:

| Polling                               | Interrupt                              |
|---------------------------------------|----------------------------------------|
| CPU continuously checks device status | CPU responds only when an event occurs |
| Wastes CPU time                       | Efficient CPU utilization              |
| Higher power consumption              | Lower power consumption                |
| Simple implementation                 | Faster response                        |

### When Interrupts Improve Performance

Interrupts are useful in:

- Motion detection
- Fire alarm systems
- Door sensors
- Ultrasonic distance measurement
- Push buttons
- Emergency shutdown systems

They provide faster response while allowing the CPU to perform other tasks.

## Answer 15:

### 1. Cost vs Accuracy

I selected the **DHT11** sensor instead of DHT22.

- Lower cost
- Lower accuracy



This trade-off is acceptable for simple monitoring applications.

---

## 2. Power Consumption vs Update Speed

The ESP32 uploads data every minute instead of every second.

- Lower battery consumption
- Slightly slower updates

Suitable for battery-operated systems.

---

## 3. Simplicity vs Expandability

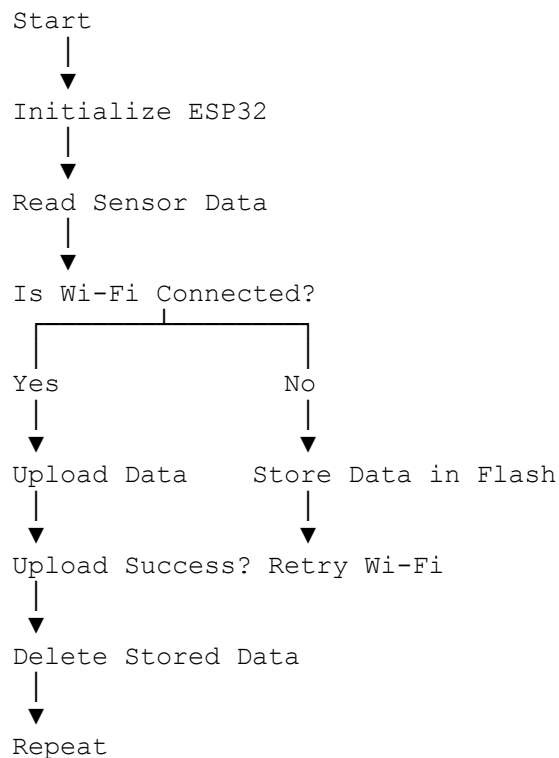
I used a single ESP32 instead of multiple controllers.

- Simpler design
- Lower cost
- Limited GPIO availability

This trade-off reduces complexity while meeting project requirements.

---

## Answer 16:

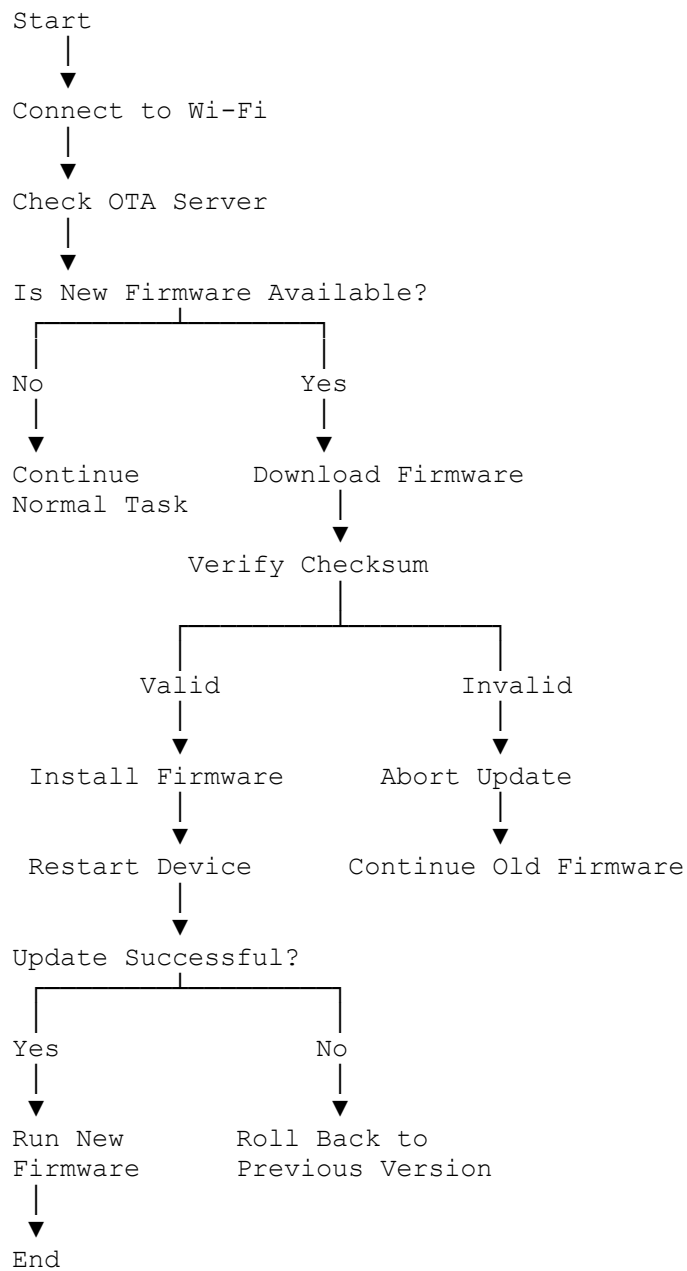


## Programs/Protocols Used

- ESP32 Firmware
- Arduino IDE
- Wi-Fi
- HTTP or MQTT
- Flash Memory/SD Card
- Cloud Database (Firebase, AWS, etc.)

---

## Answer 17:



## Error Handling

- Verify firmware checksum before installation.
- If download fails, retry after a delay.
- If installation fails, restore the previous firmware (rollback).
- Notify the user about update success or failure through the mobile application.